

Digital Room Allocation

Seminar Report Submitted in Partial Fulfilment
of the Requirements for the Degree of

Bachelor of Engineering *in* **Computer Science & Engineering**

Submitted by

ASAN ALI: (Roll No. 23UCSE4008)

Under the Supervision of

NC BARWAR
HEAD OF DEPARTMENT



Department of Computer Science and Engineering
MBM University, Jodhpur
May, 2026

CERTIFICATE

This is to certify that the seminar report entitled "Digital Room Allocation" with project implementation titled "Hostel Management System" has been submitted by Asan Ali (Roll No. 23UCSE4008) to the Department of Computer Science and Engineering, MBM University, Jodhpur, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering for the academic year 2025-2026.

The work contained herein is original and has been carried out under our supervision and guidance. This work has not been submitted elsewhere for the award of any other degree or diploma. The seminar work, in our opinion, has reached the standard fulfilling the requirements for the degree of Bachelor of Technology in Computer Science and Engineering in accordance with the regulations of the University.

Dr. NC Barwar
Head of Department (CSE)
(Supervisor)

DECLARATION

I, Asan Ali, hereby declare that this seminar report titled "**Digital Room Allocation**" with project implementation "**Hostel Management System**" is a record of original work done by me under the supervision and guidance of Dr. NC Barwar, Head of Department, Department of Computer Science and Engineering, MBM University, Jodhpur.

I further certify that this work has not formed the basis for the award of any Degree, Diploma, Associateship, Fellowship or similar recognition to any candidate of any university, and no part of this report has been reproduced as it is from any other source without proper acknowledgement and citation.

All the information presented in this report is true and accurate to the best of my knowledge and belief.

Asan Ali

Roll No.: 23UCSE4008

ACKNOWLEDGEMENT

It is a great pleasure for me to express my profound sense of gratitude and indebtedness to all those who have helped, guided, and inspired me throughout the completion of this seminar report. Without their help and cooperation, it would have been difficult to complete this task within the stipulated time frame.

I express my deepest gratitude to my seminar guide, Dr. NC Barwar, HOD, Department of Computer Science and Engineering, MBM University, Jodhpur, for his/her continuous guidance, valuable suggestions, and constant encouragement during the preparation and presentation of this seminar. His expert knowledge, constructive criticism, and positive approach have been a constant source of inspiration and motivation throughout the course of this work.

I would like to sincerely thank Dr. NC Barwar, Head of the Department of Computer Science and Engineering, MBM University, Jodhpur, for providing all the necessary facilities and for the encouraging academic environment that made this work possible. The department's infrastructure and resources have been invaluable in the execution of this project.

I am also grateful to all the faculty members and staff of the Department of Computer Science and Engineering for their direct and indirect support, constructive feedback, and technical inputs at various stages of this seminar. Their guidance has been immensely helpful in shaping the content and quality of this report.

I would like to extend my heartfelt thanks to my classmates and friends who provided moral support, shared their knowledge, and helped me during the research and development of the project. Their collaboration and encouragement have been a great source of strength.

Finally, I am deeply thankful to MBM University, Jodhpur, for providing an excellent academic environment, library resources, computing facilities, and all other necessary infrastructure that enabled me to carry out this work effectively and present it in this comprehensive form.

ABSTRACT

The Hostel Management System is a comprehensive web-based application designed to automate and digitize the complete room allocation and hostel administration process in educational institutions. This seminar report presents an in-depth study of Digital Room Allocation as the core concept, exploring the challenges of traditional manual allocation methods and proposing an efficient, transparent, and fair digital solution.

The proposed system addresses the critical operational needs of hostel administrators by providing a merit-based automatic room allocation engine. The system generates a merit list of students based on their gender and academic year, ensuring a structured and equitable allocation order. Students ranked first in the merit list receive priority room assignments, beginning with the first available rooms, creating a transparent and objectively governed allocation hierarchy.

Key functionalities of the system include: a merit-list generation module that evaluates and ranks students based on gender segregation and year of study; an automatic room allocation engine that assigns rooms to students strictly following the merit rank order; an administrator panel that enables complete manual override capability, allowing admins to manually assign any student to any available room; a room inventory management module through which administrators can configure and modify the total number of rooms, room types, and room capacities; and a student dashboard that provides real-time visibility into allocation status.

The system is developed using modern web technologies including React.js for the frontend user interface, Node.js with Express.js for the backend REST API, and MongoDB as the persistent data store. The architecture follows the MVC (Model-View-Controller) design pattern, ensuring scalability, maintainability, and clean separation of concerns.

This report is organized into five comprehensive chapters: Introduction, Requirement Modelling, System Design, Project Implementation, and Conclusion and Future Work. Each chapter methodically documents the analysis, design decisions, and technical implementations undertaken during this project, providing a thorough academic and technical record of the work accomplished.

TABLE OF CONTENTS

Chapter 1: Introduction	1
1.1 Introduction to the Seminar Topic	1
1.2 Project Description	3
1.3 Problem Statement and Motivation	5
1.4 Project Objectives and Scope	7
1.5 Key Features and Modules	9
Chapter 2: Requirement Modelling	12
2.1 Functional Requirements	12
2.2 Non-Functional Requirements	14
2.3 Use Case Analysis	16
2.4 Data Flow Analysis	18
2.5 System Actors and Interactions	20
Chapter 3: System Design	22
3.1 System Architecture Overview	22
3.2 Database Design	24
3.3 Module-Level Design	27
3.4 Merit List Algorithm Design	29
3.5 API Design	31
Chapter 4: Project Implementation	33
4.1 Technology Stack	33
4.2 Implementation of Merit List Module	35
4.3 Automatic Room Allocation Engine	37
4.4 Admin Manual Allocation Module	39
4.5 Room Inventory Management	41
4.6 Testing and Validation	43
Chapter 5: Conclusion and Future Work	46
5.1 Summary of Work Done	46
5.2 Achievements and Findings	47
5.3 Limitations	48
5.4 Future Enhancements	49
References	51

LIST OF FIGURES

Fig 1.1 – First Page Of Hostel Management System	14
Fig 1.2 – Login/Register Page	15
Fig 2.1 – ER Diagram -Room Allocation	27
Fig 2.2 – ER Diagram – Full Diagram of HMS	27
Fig 3.1 – Schema Relationship Diagram	28
Fig 3.2 – Automatic Room Allocation Flowchart	29
Fig 3.3 – Merit List Dashboard	36
Fig 3.4 – Room Inventory Management Screen	36
Fig 3.5 – Room Allocation Page	37
Fig 4.1 – Admin Dashboard Wireframe	37

LIST OF TABLES

Table 2.1 – Functional Requirements Summary	17
Table 2.2 – Non-Functional Requirements	17
Table 2.3 – System Actors and Their Roles	18
Table 3.1 – <i>Student Collection – MongoDB Schema</i>	22
Table 3.3 – Allocation Collection – MongoDB Schema	23
Table 3.4 – <i>Sample Merit List (Male Students – All Years)</i>	25
Table 4.2 – Technology Stack Summary	30
Table 4.3 – <i>Room Types and Capacity Configuration (Sample Data)</i>	34
Table 4.4 – <i>Test Cases for Room Allocation Module</i>	35
Table 4.5 – <i>Proposed Future Enhancements</i>	39

Chapter 1: Introduction

1.1 Introduction to the Seminar Topic – Digital Room Allocation

Digital Room Allocation refers to the process of assigning accommodation or workspace to individuals or groups using a technology-driven platform, replacing the traditional paper-based or manually managed processes with automated, data-driven, and web-enabled systems. In the context of educational institutions such as universities and engineering colleges, room allocation is a fundamental administrative process that directly impacts the comfort, safety, and academic performance of residential students.

Historically, hostel room allocation in Indian universities and colleges has been conducted through manual processes that involve physical forms, notice boards, and in-person assignments. These approaches are inherently prone to human errors, favoritism, inefficiency, and a lack of transparency. Students often receive rooms without any systematic ordering, leading to dissatisfaction and disputes. Moreover, manual processes require significant administrative effort, paper resources, and time — resources that could be saved through digitization.

The concept of Digital Room Allocation brings transformative change to this domain. By leveraging web-based platforms, relational or NoSQL databases, and algorithmic decision-making, institutions can ensure that room assignments are governed by well-defined and objectively applied rules. The entire lifecycle of room allocation — from capturing student data, computing merit rankings, assigning rooms, managing inventories, and notifying students — can be handled seamlessly within a single integrated software system.

In the contemporary landscape of educational technology (EdTech), digital hostel management has become a priority for institutions aspiring to enhance their administrative efficiency. Leading universities across India, including IITs, NITs, and central universities, have implemented or are in the process of implementing digital hostel management systems. These systems vary in complexity and features, but they share a common goal: to make the room allocation process faster, fairer, and more transparent.

The scope of digital room allocation encompasses several dimensions. First, it involves the creation of a merit-based ranking system that evaluates students according to pre-determined criteria — in this project, the primary criteria are gender and year of study — and arranges them in a ranked list. This merit list determines the order in which students receive room

assignments, ensuring that senior students or those with higher priority are accommodated first. Second, the system must provide administrative controls that allow hostel wardens or registrar-level administrators to intervene manually when necessary — for example, when a student has special needs or when a merit-based assignment needs to be overridden due to institutional reasons.

Third, digital room allocation requires robust backend support for room inventory management. Administrators must be able to define the total number of rooms, configure room types (single occupancy, double occupancy, triple occupancy), assign room numbers, and modify capacity settings at any time. This flexibility ensures that the system can adapt to changes in hostel infrastructure without requiring code-level modifications.

The significance of digital room allocation extends beyond administrative convenience. When students can view their allocation status online, track the merit list, and receive automated notifications, it significantly reduces the anxiety and confusion that often surrounds the allocation season. A well-implemented digital allocation system fosters trust between the institution and its students, demonstrating a commitment to transparency and procedural fairness.

1.2 Project Description

The project titled "Hostel Management System" is a full-stack web application designed to digitize and automate the complete hostel room allocation process at MBM University, Jodhpur. The system serves as a practical implementation of the Digital Room Allocation concept, providing a unified platform for both administrators and students to interact with the hostel management processes.

The Hostel Management System is designed with a dual-user model: the Administrator and the Student. Each user category has distinct roles, permissions, and interfaces within the system. The administrator is the primary decision-maker who controls all aspects of the hostel management process, including configuring room inventories, managing student records, generating merit lists, and performing or approving room allocations. The student, on the other hand, is a passive user who can view their allocation status, check the merit list, and update their personal information as required.

At its technical core, the system is a three-tier web application comprising: (1) a React.js-based single-page application (SPA) that serves as the user interface, (2) a Node.js and Express.js powered RESTful API backend that handles all business logic, and (3) a MongoDB database

that stores and manages all persistent data including student profiles, room inventories, merit lists, and allocation records.

The project's primary innovation lies in its merit-based automatic allocation engine. When the administrator triggers the automatic allocation process, the system reads the pre-computed merit list — organized by gender and year — and assigns rooms in the order of merit rank. The student ranked first receives the first available room, the student ranked second receives the second, and so on. Once the automatic allocation is exhausted or paused, the administrator can use the manual allocation interface to assign specific rooms to remaining or unallocated students.

The system maintains complete audit trails of all allocation activities. Every manual override and every automatic assignment is logged with a timestamp and the identity of the administrator who performed the action. This ensures accountability and traceability, which are essential qualities of any institutional management system.

1.3 Problem Statement and Motivation

The problem of hostel room allocation at educational institutions like MBM University is multifaceted and deeply rooted in administrative inefficiency. Currently, the allocation process involves manual form-filling by students, physical submission of documents to the hostel office, and a largely subjective decision-making process by hostel staff. This process suffers from several critical issues:

- **Lack of Transparency:** Students are often unaware of the criteria used for room allocation. There is no publicly accessible ranking or merit list, making it difficult for students to understand why they were or were not assigned a preferred room.
- **Administrative Overload:** Hostel staff must manually process hundreds of allocation requests each semester, which is time-consuming, error-prone, and inefficient. Duplications, misplaced forms, and clerical mistakes are common occurrences.
- **Absence of Fairness Mechanism:** Without a systematic merit-based system, students from the same year and gender compete for rooms without any objective ordering mechanism, leading to conflict and dissatisfaction.
- **No Real-Time Status Updates:** Under the manual system, students must physically visit the hostel office to know their allocation status, wasting time and creating unnecessary congestion.
- **Difficulty in Room Inventory Management:** Administrators have no centralized digital record of room occupancy, availability, and capacity. Changes in hostel infrastructure are poorly documented and difficult to track.
- **Data Integrity Issues:** Paper-based records are susceptible to damage, loss, and manipulation. Digital storage with proper backup mechanisms provides far greater reliability.

The motivation for this project stems from the urgent need to address these issues. With increasing student enrollment and growing hostel capacities, the existing manual system is no longer sustainable. A digital hostel management system with intelligent room allocation capabilities will not only resolve the above problems but will also position MBM University as a technologically progressive institution.

1.4 Project Objectives and Scope

The primary objective of this project is to design and develop a Hostel Management System that automates the room allocation process using a merit-based digital framework. The specific objectives are listed below:

1. To develop a Merit List Generation Module that ranks students based on gender and year of study.
2. To implement an Automatic Room Allocation Engine that assigns rooms to students strictly following the merit order.
3. To build an Admin Manual Allocation Interface that enables administrators to override automatic allocation and manually assign rooms to specific students.
4. To create a Room Inventory Management Module through which administrators can set, modify, and monitor the number of rooms, room types, and occupancy status.
5. To develop a Student Dashboard that provides students with real-time access to their allocation status and the merit list.
6. To ensure the system is secure, scalable, and maintainable using industry-standard web development practices.
7. To validate the system through comprehensive testing including unit tests, integration tests, and user acceptance tests.

The scope of the project includes the full software development lifecycle for the described modules: requirement gathering, system design, coding, testing, and documentation. The system is scoped for use within a single hostel or group of hostels under one administrative unit. Multi-campus or multi-institution deployment is considered a future enhancement.

1.5 Key Features and Modules

The Hostel Management System incorporates the following key modules and features:

1.5.1 Merit List Generation

The merit list module is the backbone of the Digital Room Allocation system. It fetches student records from the database and applies a set of configurable ranking criteria. In the current

implementation, students are ranked based on their academic year (higher year gets priority) and gender (male and female lists are generated separately). Within the same year, students are further sub-ranked alphabetically by name as a tie-breaker. The resulting merit list is stored in the database and can be viewed by both administrators and students.

1.5.2 Automatic Room Allocation

Once the merit list is generated and rooms are configured in the inventory, the administrator can trigger the automatic room allocation. The system iterates through the merit list from rank 1 and assigns the first available room to each student. Rooms are matched by type (single/double/triple) based on pre-set preferences. The allocation continues until either all rooms are occupied or all students on the merit list have been processed. Students who could not be accommodated automatically are placed on a waiting list.

1.5.3 Admin Manual Room Allocation

This module provides the administrator with full manual control over the allocation process. Through an intuitive dashboard interface, the admin can search for any student, view available rooms, and directly assign a room. This feature is particularly useful for accommodating students with special requirements, resolving appeals, or assigning rooms to students who were not covered by the automatic allocation run.

1.5.4 Room Quantity and Type Configuration

The administrator can dynamically configure the hostel's room inventory. This includes adding new rooms, modifying the capacity (number of beds) of existing rooms, setting room types, and deactivating rooms that are under maintenance. The system provides a real-time dashboard showing total rooms, occupied rooms, available rooms, and rooms under maintenance.

1.5.5 Student Registration and Profile Management

Students can register themselves or be registered by the administrator. Each student profile includes their name, roll number, gender, year of study, branch, contact information, and profile photograph. The system uses this data during merit list generation and allocation processing.

The Next Generation Hostel Management

A premium, intelligent platform to automate room allocations,
resolve complaints faster, and unify campus living.

500+

Students

120

Rooms

100%

Digital

24/7

Support

Welcome Back

Log in to your HMS account

Student

Admin

Email Address

✉ asan1@gmail.com

Password

🔒

Sign In

Don't have an account? [Register here](#)

Chapter 2: Requirement Modelling

2.1 Functional Requirements

Functional requirements describe the specific behaviors, features, and functions that the Hostel Management System must support. These requirements are derived from extensive stakeholder analysis involving hostel administrators, students, and faculty members. The functional requirements are categorized into three groups: Admin Requirements, Student Requirements, and System-Level Requirements.

2.1.1 Admin Requirements

- The admin shall be able to log in to the system using a unique username and password.
- The admin shall be able to view, add, edit, and delete student records.
- The admin shall be able to configure the total number of rooms available in the hostel.
- The admin shall be able to set room types (single, double, triple), room numbers, and floor details.
- The admin shall be able to generate a merit list of students filtered by gender and year.
- The admin shall be able to trigger the automatic room allocation process.
- The admin shall be able to manually allocate any unallocated room to any unallocated student.
- The admin shall be able to cancel or modify an existing room allocation.
- The admin shall be able to view the current occupancy report of the hostel.
- The admin shall be able to generate and export allocation reports in PDF format.

2.1.2 Student Requirements

- The student shall be able to register and create a personal profile.
- The student shall be able to log in and view their room allocation status.
- The student shall be able to view the current merit list (their own rank and others).
- The student shall receive an automated notification (on-screen alert) upon room assignment.
- The student shall be able to view room details including room number, floor, room type, and roommates.

2.1.3 System-Level Requirements

- The system shall automatically sort and rank students when the merit list is generated.
- The system shall prevent double-allocation of a room that is already at full capacity.
- The system shall log all allocation and de-allocation activities with timestamps.
- The system shall maintain separate merit lists for male and female students.

Table 2.1: Functional Requirements Summary

Req ID	Requirement	Priority	Type
FR-01	Admin Login and Authentication	High	Functional
FR-02	Student Registration and Profile	High	Functional
FR-03	Merit List Generation by Gender & Year	High	Functional
FR-04	Automatic Room Allocation (Merit-Based)	High	Functional
FR-05	Manual Room Allocation by Admin	High	Functional
FR-06	Room Inventory Configuration	High	Functional
FR-07	Allocation Status View for Students	Medium	Functional
FR-08	Allocation Cancellation and Modification	Medium	Functional
FR-09	Occupancy Report Generation	Medium	Functional
FR-10	Activity Log and Audit Trail	Low	Functional

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational characteristics of the system. These requirements govern how the system performs its functions rather than what those function

Table 2.2: Non-Functional Requirements

Quality Attribute	Requirement Description	Metric / Target
Performance	System should handle concurrent users without degradation	Response time < 2 sec for 100 users
Security	All passwords must be hashed; sessions must be token-based	JWT + bcrypt encryption
Usability	Interface should be intuitive for non-technical admin staff	Task completion rate > 90%
Reliability	System uptime should be maximized during allocation season	Availability > 99.5%
Scalability	System should support addition of new hostels and rooms	Horizontal scalability via MongoDB
Maintainability	Code should follow modular MVC architecture	Each module < 300 lines
Portability	System should run on any standard browser	Chrome, Firefox, Edge support

Quality Attribute	Requirement Description	Metric / Target
Data Integrity	No duplicate allocations should be possible	Unique constraint on allocation

2.3 Use Case Analysis

Use Case Analysis models the interactions between system actors and the system itself. The primary actors in the Hostel Management System are: the Admin (Hostel Administrator / Warden) and the Student. A secondary actor is the System itself, which performs automated operations like sorting and allocation.

2.3.1 Actor Descriptions

Table 2.3: System Actors and Their Roles

Actor	Role Description	Access Level
Admin	Manages all hostel operations, room configuration, student data, and allocation processes	Full Access – Read/Write/Delete
Student	Views merit list, allocation status, and personal profile information	Read-Only (own data + merit list)
System (Auto)	Executes merit list sorting and automatic allocation on admin command	Internal System Process

2.3.2 Key Use Cases

UC-01: Generate Merit List

- Actor: Admin
- Description: The admin selects gender and year parameters and triggers merit list generation. The system fetches all eligible students and ranks them.
- Precondition: Student records must be present in the database.
- Post condition: A ranked merit list is stored and displayed.

UC-02: Automatic Room Allocation

- Actor: Admin, System
- Description: Admin triggers automatic allocation. System reads merit list and assigns rooms in rank order.
- Precondition: Merit list must be generated; rooms must be configured.
- Post condition: Eligible students are assigned rooms; remaining students are waitlisted.

UC-03: Manual Room Allocation

- Actor: Admin
- Description: Admin searches for an unallocated student and selects an available room for direct assignment.
- Precondition: Student must be unallocated; room must be available.
- Post condition: Student is allocated to the selected room; room status updates.

UC-04: Configure Room Inventory

- Actor: Admin
- Description: Admin adds, edits, or deactivates room records, modifying room count and capacity.
- Precondition: Admin is authenticated.
- Post condition: Room inventory is updated and reflected in the system.

2.4 Data Flow Analysis

Data Flow Diagrams (DFDs) represent the flow of data through the system. The Level-0 DFD shows the system as a single process interacting with its external entities. The Level-1 DFD decomposes the central system into its major sub-processes.

2.4.1 Level-0 DFD (Context Diagram)

The context diagram identifies two main external entities: Admin and Student. The Admin inputs data such as room configurations, student records, and allocation commands. The system outputs allocation confirmations, merit lists, and occupancy reports to the Admin. The Student provides registration data and receives allocation status and merit rank information from the system.

2.4.2 Level-1 DFD (Main Processes)

At Level 1, the system is decomposed into five major processes: (1) User Authentication, which validates credentials and generates session tokens; (2) Student Data Management, which handles CRUD operations on student records; (3) Merit List Generation, which queries and ranks students; (4) Room Allocation Engine, which processes both automatic and manual assignments; and (5) Room Inventory Management, which maintains room configuration data.

Each process communicates with specific data stores: the Student Database, the Room Inventory Database, the Merit List Store, and the Allocation Log. Data flows between these processes are clearly defined, ensuring that inputs and outputs are unambiguous and traceable.

2.5 System Actors and Interactions

The interaction between system actors is best understood through a detailed examination of the administrator workflow and the student workflow. The administrator workflow begins with login authentication, proceeds to student data management, then to merit list generation, followed by automatic or manual allocation, and finally to reporting. The student workflow, by contrast, is simpler: login, view merit rank, and view allocation status.

These two workflows intersect at the merit list and allocation status data stores, which are accessible to both actors but with different permissions. The admin can modify these records; the student can only read them. This role-based access control model is fundamental to the system's security architecture.

Chapter 3: System Design

3.1 System Architecture Overview

The Hostel Management System follows a classic three-tier architecture, which separates concerns into distinct layers: the Presentation Layer, the Business Logic Layer, and the Data Layer. This architectural pattern is widely adopted in enterprise web application development due to its inherent benefits of separation of concerns, scalability, testability, and maintainability.

3.1.1 Presentation Layer (Frontend)

The presentation layer is implemented using React.js, a modern JavaScript library for building user interfaces. The frontend is structured as a Single-Page Application (SPA) that communicates with the backend exclusively through RESTful API calls. React's component-based architecture enables the development of reusable UI components, making the codebase modular and maintainable.

Key frontend components include the Admin Dashboard, Merit List View, Room Allocation Panel, Room Inventory Manager, Student Profile Page, and Login/Authentication Module. Each component manages its own state using React's `useState` and `useEffect` hooks. Global state such as authentication tokens and user role information is managed using the Context API or Redux (for larger state management needs).

3.1.2 Business Logic Layer (Backend)

The backend is built using Node.js with the Express.js framework. Express provides a lightweight and flexible platform for building RESTful APIs, handling HTTP requests, executing middleware functions, and routing API calls to the appropriate controllers. The backend follows the MVC (Model-View-Controller) design pattern: Models define the data schemas (using Mongoose ODM), Controllers implement the business logic, and Routes map HTTP endpoints to controller functions.

The merit list generation and automatic allocation algorithms are implemented as service-level functions in the backend. These services are invoked by the allocation controller when the admin triggers the respective operations. The algorithms are designed to be stateless and side-effect-free, ensuring that they produce consistent results when given the same input data.

3.1.3 Data Layer (Database)

MongoDB is used as the primary database. As a document-oriented NoSQL database, MongoDB provides schema flexibility, horizontal scalability, and native support for JSON-like documents, making it an ideal choice for a dynamic application like the Hostel Management System. Data is organized into three primary collections: students, rooms, and allocations. A fourth collection, meritlists, stores the generated merit list snapshots.

3.2 Database Design

The database design for the Hostel Management System consists of four primary collections in MongoDB. The following table describes the schema of the Student collection

Table 3.1: Student Collection – MongoDB Schema

Field Name	Data Type	Description	Constraints
id	ObjectId	Unique identifier (auto-generated)	Primary Key
name	String	Full name of the student	Required, Indexed
rollNo	String	University roll number	Required, Unique
gender	String (enum)	Gender: 'Male' or 'Female'	Required
year	Number (1-4)	Current academic year	Required
branch	String	Department/Branch of study	Required
email	String	Student email address	Required, Unique
phone	String	Mobile phone number	Optional
address	String	Permanent home address	Optional
isAllocated	Boolean	True if room has been assigned	Default: false
createdAt	Date	Profile creation timestamp	Auto-generated

Field Name	Data Type	Description	Constraints
_id	ObjectId	Unique identifier (auto-generated)	Primary Key
roomNumber	String	Room identifier (e.g., A-101)	Required, Unique
floor	Number	Floor number in the hostel building	Required

Field Name	Data Type	Description	Constraints
type	String (enum)	Room type: Single/Double/Triple	Required
capacity	Number	Maximum number of occupants	Required
currentOccupancy	Number	Current number of allocated students	Default: 0
isAvailable	Boolean	True if room has vacancy	Computed
isUnderMaintenance	Boolean	True if room is out of service	Default: false
hostelBlock	String	Block/Wing identifier (e.g., Block A)	Required
amenities	Array<String>	List of facilities (AC, fan, etc.)	Optional

Table 3.3: Allocation Collection – MongoDB Schema

Field Name	Data Type	Description	Constraints
id	ObjectId	Unique identifier	Primary Key
studentId	ObjectId (ref)	Reference to Student collection	Required, Indexed
roomId	ObjectId (ref)	Reference to Room collection	Required, Indexed
allocationType	String	"Auto" or "Manual"	Required
allocatedBy	ObjectId (ref)	Reference to admin who performed it	Required
allocationDate	Date	Timestamp of allocation event	Auto-generated
status	String (enum)	Active / Cancelled / Transferred	Default: Active
remarks	String	Admin notes or remarks	Optional

3.3 Module-Level Design

The system is composed of six functional modules, each encapsulating a specific area of functionality. The modules are designed with high cohesion (all related logic is within the module) and low coupling (minimal dependencies between modules), which are hallmarks of good software engineering practice.

3.3.1 Authentication Module

This module handles user registration, login, and session management. It uses bcrypt for password hashing and JSON Web Tokens (JWT) for stateless session management. Upon

successful login, the server issues a JWT that the client stores in memory and includes in the Authorization header of all subsequent API requests. The token includes the user's ID, role (admin or student), and expiry time.

3.3.2 Student Management Module

This module provides CRUD (Create, Read, Update, Delete) operations for student records. Only administrators can create, update, or delete student records. Students can view their own profiles. The module also handles bulk import of student data via CSV file upload, which is useful at the start of each academic year when large numbers of students need to be registered.

3.3.3 Merit List Module

The merit list module is responsible for generating and storing ranked student lists. The generation algorithm takes gender and year as input parameters, queries the student collection, applies the ranking logic, and saves the resulting list as a meritlist document in the database. The algorithm sorts students by year in descending order (4th year > 3rd > 2nd > 1st) and then alphabetically by name within the same year.

3.3.4 Room Allocation Module

This is the most complex module in the system. It contains two sub-modules: the Automatic Allocation Engine and the Manual Allocation Interface. The Automatic Engine iterates through the merit list and attempts to assign a room to each student using a best-fit allocation strategy. The Manual Interface provides the administrator with a search and assign capability, with real-time availability checks to prevent over-allocation.

3.3.5 Room Inventory Module

This module manages the room database. Administrators can add new rooms, edit room details, mark rooms as under maintenance, and view the current occupancy dashboard. The module also provides aggregate statistics including total rooms, occupied rooms, available rooms, and maintenance rooms.

3.4 Merit List Algorithm Design

The merit list generation algorithm is a core intellectual contribution of this project. The algorithm ensures that room allocation is performed in a fair, consistent, and transparent manner. The algorithm operates as follows:

8. Query the student collection, filtering by the given gender (e.g., 'Male' or 'Female') and isAllocated: false.
9. Sort the resulting student list in descending order of academic year (year 4 > 3 > 2 > 1).
10. Within each year group, sort students alphabetically by their full name as a secondary tie-breaker.
11. Assign sequential ranks to the sorted list (Rank 1 to Rank N).
12. Store the ranked list as a new MeritList document in the database, tagged with gender, generation timestamp, and academic semester.
13. Return the ranked list to the frontend for display.

The algorithm ensures that senior students (higher academic year) are always given priority over junior students, reflecting the institutional policy of recognizing seniority. The alphabetical sub-ordering within the same year ensures that ties are resolved in a neutral, non-discriminatory manner.

Table 4.2: Sample Merit List (Male Students – All Years)

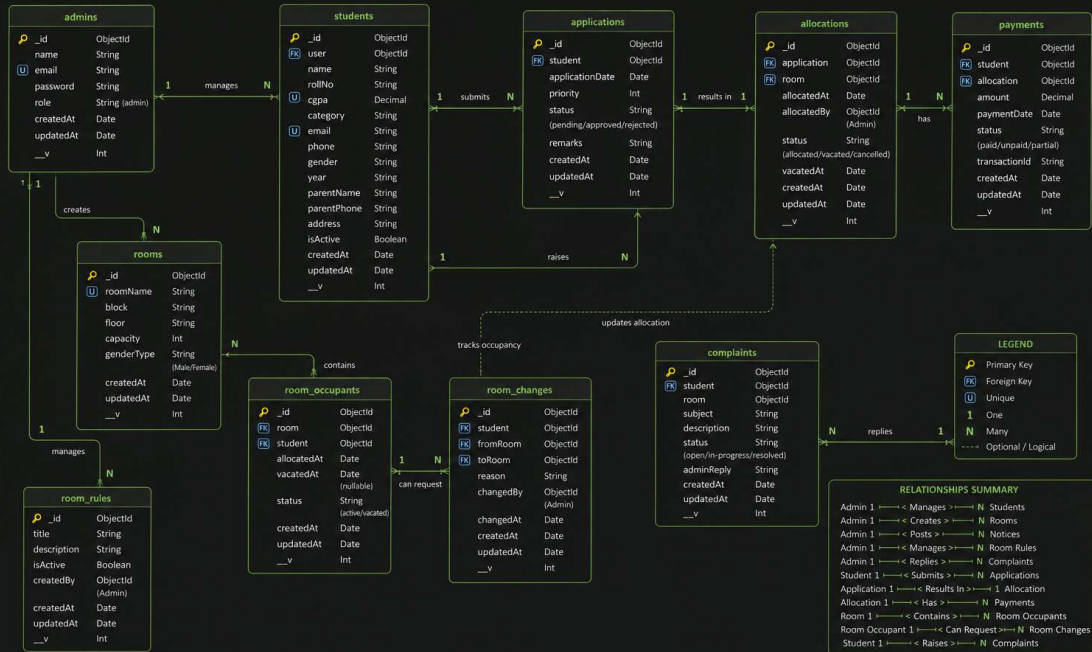
Merit Rank	Student Name	Gender	Year	Branch	Roll No.
1	Aarav Mehta	Male	4th Year	CSE	20UCSE4001
2	Abhishek Singh	Male	4th Year	CSE	20UCSE4002
3	Rajan Patel	Male	4th Year	ECE	20UECE4005
4	Vikram Tiwari	Male	3rd Year	CSE	21UCSE4010
5	Yash Sharma	Male	3rd Year	ME	21UME4003
6	Anuj Verma	Male	2nd Year	CSE	22UCSE4007
7	Deepak Kumar	Male	2nd Year	IT	22UIT4002
8	Harsh Gupta	Male	1st Year	CSE	23UCSE4011

3.5 API Design

The backend exposes a RESTful API organized around the primary resources of the system. The API follows REST principles: resources are identified by URIs, operations are performed using HTTP verbs (GET, POST, PUT, DELETE), and responses use standard HTTP status codes. Authentication is enforced using JWT middleware on all protected endpoints.

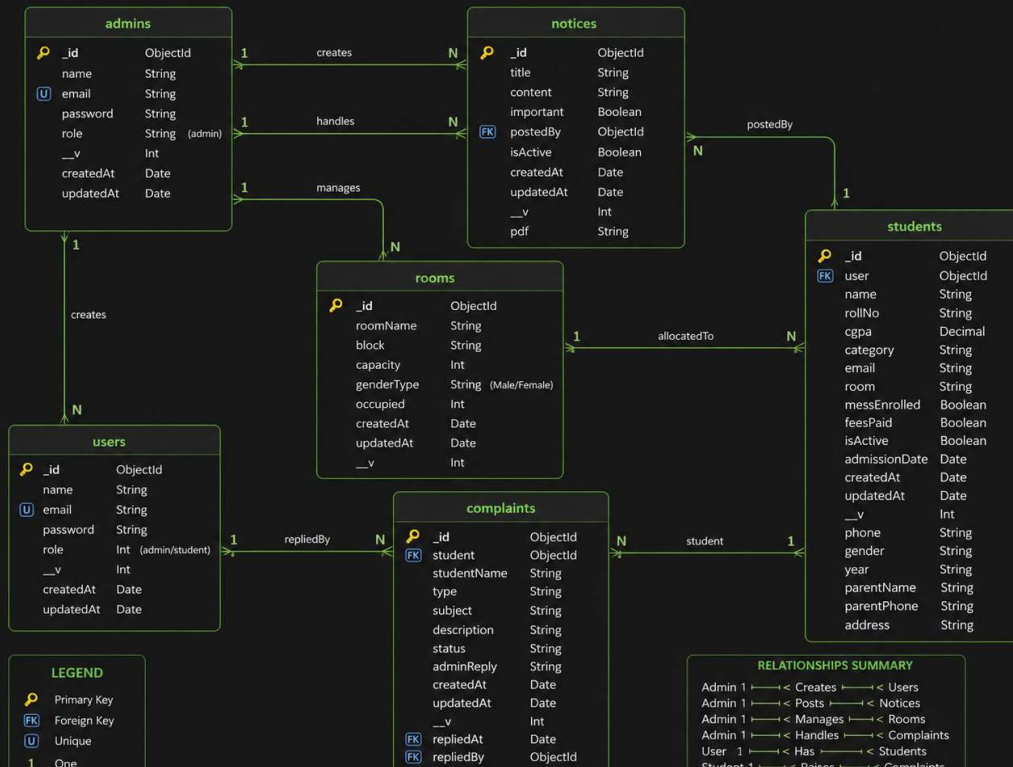
HTTP Method	Endpoint	Description	Auth Required
POST	/api/auth/login	Admin or student login	No
POST	/api/auth/register	Student registration	No
GET	/api/students	Get all student records	Admin
POST	/api/students	Add a new student record	Admin
PUT	/api/students/:id	Update student information	Admin
DELETE	/api/students/:id	Delete a student record	Admin
GET	/api/merit-list	Get generated merit list	Both
POST	/api/merit-list/generate	Generate new merit list	Admin
GET	/api/rooms	Get all room details	Admin
POST	/api/rooms	Add a new room to inventory	Admin
PUT	/api/rooms/:id	Update room details or capacity	Admin
POST	/api/allocations/auto	Trigger automatic allocation	Admin
POST	/api/allocations/manual	Manual room assignment	Admin
GET	/api/allocations/student/:id	Get allocation for student	Both
DELETE	/api/allocations/:id	Cancel/revoke an allocation	Admin

DIGITAL ROOM ALLOCATION SYSTEM – ER DIAGRAM

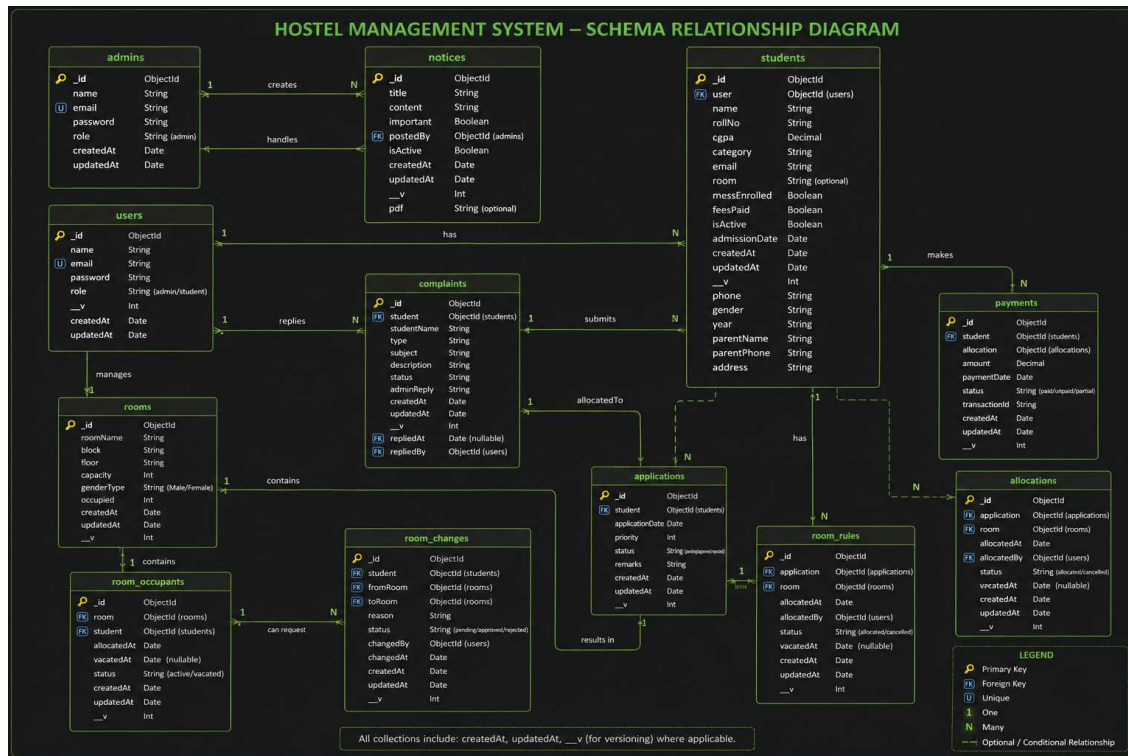


This ER diagram covers the complete Digital Room Allocation System including room creation, student applications, allocation, occupancy tracking, payments, complaints, notices and admin management.

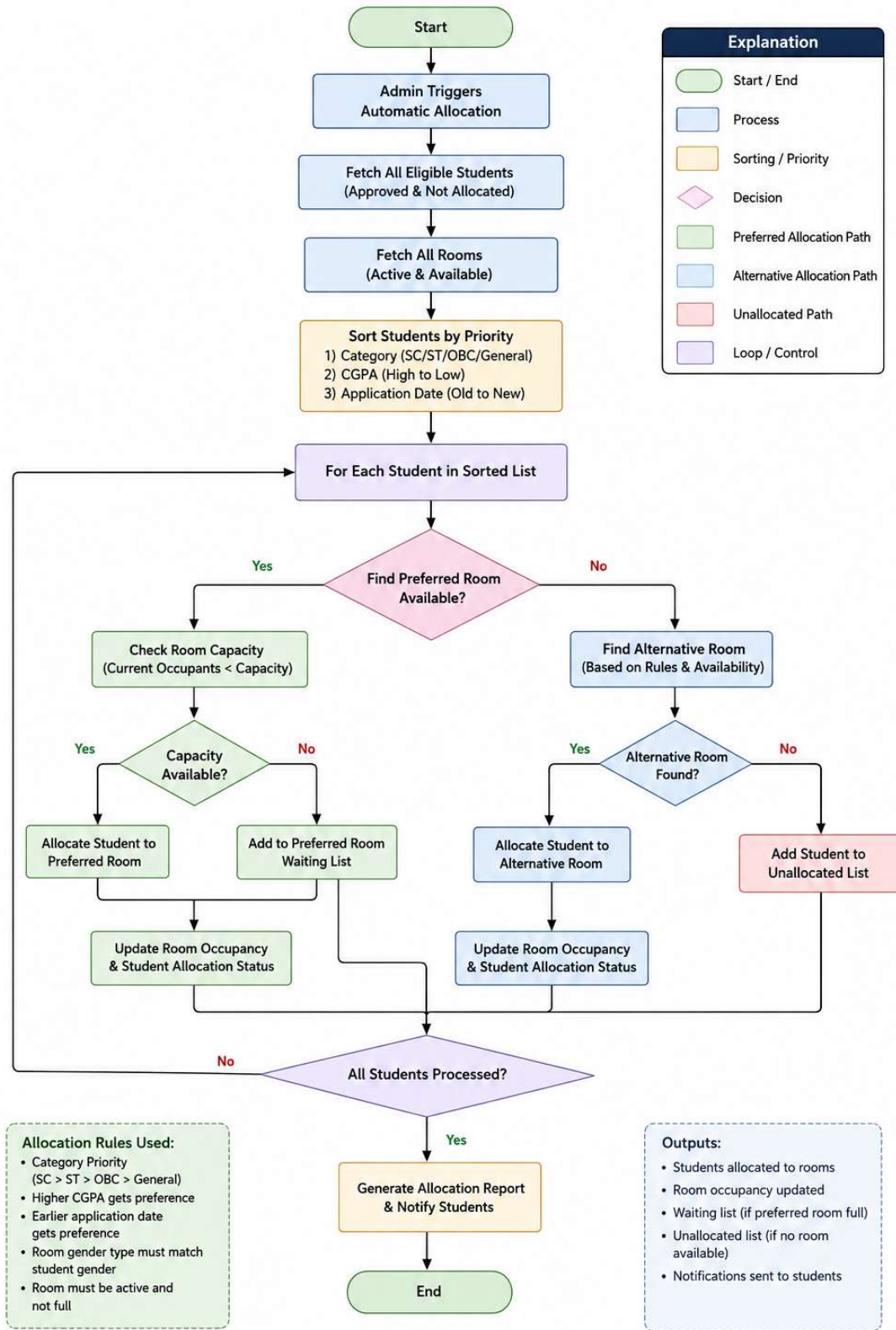
HOSTEL MANAGEMENT SYSTEM – ER DIAGRAM



All collections include: createdAt, updatedAt, __v (for versioning) where applicable.



Automatic Room Allocation Flowchart



Chapter 4: Project Implementation

4.1 Technology Stack

The Hostel Management System is built using the MERN stack (MongoDB, Express.js, React.js, Node.js), which is one of the most popular and widely adopted technology stacks for full-stack web application development. Each component of the stack was chosen based on its suitability for the project requirements, developer community support, and long-term maintainability.

Table 4.1: Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Component-based SPA user interface
Frontend	Tailwind CSS	3.x	Utility-first CSS styling framework
Frontend	Axios	1.x	HTTP client for API communication
Backend	Node.js	20.x LTS	JavaScript runtime for server-side code
Backend	Express.js	4.x	Minimalist web framework for REST APIs
Backend	Mongoose	7.x	ODM for MongoDB schema and queries
Backend	JSON Web Token	9.x	Stateless authentication tokens
Backend	bcryptjs	2.x	Password hashing library
Database	MongoDB	7.x	NoSQL document database
Tools	Postman	Latest	API testing and documentation
Tools	Git + GitHub	Latest	Version control and collaboration
Tools	VS Code	Latest	Integrated development environment

4.2 Implementation of the Merit List Module

The merit list module is implemented as a combination of a backend service function and a React frontend component. The backend service, `meritListService.js`, handles all data processing logic, while the React component `MeritListPanel.jsx` renders the results in a tabular format.

4.2.1 Backend Implementation

The merit list generation service follows these implementation steps. First, it accepts the gender and year (optional) parameters from the HTTP request body. If year is not specified, the service generates a comprehensive merit list across all years for the specified gender. It then queries the MongoDB students collection using Mongoose's `find()` method with a filter object that matches the requested gender and unallocated status.

The query results are sorted using Mongoose's `sort()` method with a compound sort key: `{ year: -1, name: 1 }`. The `-1` for year ensures descending order (4th year first), and `1` for name ensures ascending alphabetical order as the tie-breaker. The sorted array is then mapped to add a rank property, with each element's rank being its 1-based index position in the array.

The ranked array is then saved to the meritlists collection as a new document, with metadata including the generation timestamp, the admin's user ID, the gender filter, and the semester information. The document is then returned as the API response.

4.2.2 Frontend Display

The MeritListPanel React component fetches the merit list data from the API and renders it in a styled HTML table. The table columns include Rank, Student Name, Roll Number, Year, Branch, and Allocation Status. The component uses React's `useState` to manage the list data and filter states, and `useEffect` to trigger the API call when the component mounts or when the gender filter changes.

An important UI feature is the color-coded allocation status indicator: students who have already been allocated a room are shown with a green badge, while unallocated students show an orange badge. This gives the admin a quick visual overview of the allocation progress.

4.3 Automatic Room Allocation Engine

The automatic allocation engine is the most algorithmically significant component of the system. It is invoked by a POST request to `/api/allocations/auto` and executes a deterministic, sequential allocation algorithm.

4.3.1 Algorithm Logic

The engine begins by fetching the most recently generated merit list for the specified gender. It then retrieves all available rooms (where `currentOccupancy < capacity` and

isUnderMaintenance is false) from the rooms collection, sorted by room number in ascending order.

The engine then iterates through the merit list from rank 1. For each student, it searches the available rooms list for a room with vacancy. If found, it creates a new allocation document with allocationType set to 'Auto', updates the room's currentOccupancy, and updates the student's isAllocated flag to true. If no room is available, the student is added to a waitlist array.

After processing all merit-listed students, the engine returns a summary response containing: total students processed, students allocated, students waitlisted, and an array of the created allocation records. This summary is displayed to the admin in the dashboard upon completion.

4.3.2 Handling Edge Cases

Several edge cases are handled by the engine: (1) If no merit list exists for the specified gender, the API returns a 400 error with an appropriate message. (2) If no rooms are available, all students are waitlisted and the admin is notified. (3) If a student is already allocated (isAllocated: true), the engine skips that student and continues to the next. (4) If the room reaches full capacity during the allocation run, it is removed from the available rooms pool for subsequent allocations.

The engine uses MongoDB transactions to ensure atomicity: either all operations in a single allocation event (creating the allocation document, updating room occupancy, updating student status) succeed together, or all fail and roll back. This prevents partial updates that could leave the database in an inconsistent state.

4.4 Admin Manual Allocation Module

The manual allocation module provides the administrator with direct, granular control over room assignments. This feature is designed for situations where the automatic algorithm is insufficient — for example, when a student has a disability and requires a ground-floor room, or when two specific students have requested to be roommates.

4.4.1 Admin Interface

The manual allocation interface is presented as a two-panel layout on the admin dashboard. The left panel displays a searchable list of unallocated students, showing their name, roll

number, year, and gender. The right panel displays a list of available rooms, showing the room number, type, capacity, current occupancy, and floor.

The admin selects a student from the left panel and a room from the right panel, then clicks the 'Allocate' button. The system immediately validates the selection: it checks that the student is indeed unallocated and that the selected room has vacancy. If both validations pass, the allocation is processed. If either fails, an error message is displayed and no database changes are made.

4.4.2 Backend Processing

The manual allocation request is routed to the allocation controller's `manualAllocate` function. This function first checks the validity of the provided `studentId` and `roomId` by querying the database. It then verifies that the student's `isAllocated` field is false and that the room's `currentOccupancy` is less than its capacity. If all checks pass, it creates a new allocation document with `allocationType`: 'Manual' and the current admin's user ID as `allocatedBy`.

The function updates the room's `currentOccupancy` and the student's `isAllocated` flag within a single MongoDB transaction. It then returns the allocation record to the frontend, which updates its local state to remove the student from the unallocated list and the room from the available rooms list if it is now full.

4.5 Room Inventory Management

The room inventory management module enables administrators to maintain an accurate and up-to-date record of all hostel rooms. This module is critical because the accuracy of the room allocation process depends entirely on the accuracy of the room inventory.

4.5.1 Room Configuration Features

The admin can perform the following operations through the room inventory module:

- **Add New Room:** The admin enters the room number, floor, hostel block, room type (single/double/triple), and maximum capacity. The system creates a new room document with `currentOccupancy` initialized to 0 and `isAvailable` set to true.
- **Edit Room:** The admin can modify the capacity, room type, or amenities of an existing room. If the new capacity is less than the current occupancy, the system rejects the change and displays an error.

- **Set Room as Under Maintenance:** The admin can toggle a room's `isUnderMaintenance` flag. Maintenance rooms are excluded from the allocation pool until marked as available again.
- **View Room Dashboard:** A real-time dashboard displays summary statistics including total rooms, available rooms, occupied rooms, rooms under maintenance, and percentage occupancy.

Table 4.4: Room Types and Capacity Configuration (Sample Data)

Room Type	Capacity	Total Rooms (Example)	Current Occupancy	Available
Single	1	20	14	6
Double	2	50	72 / 100	28 rooms partial
Triple	3	30	60 / 90	15 rooms partial
Under Maintenance	N/A	5	N/A	0

4.6 Testing and Validation

A comprehensive testing strategy was employed to validate the correctness, reliability, and performance of the Hostel Management System. Testing was conducted at three levels: unit testing, integration testing, and user acceptance testing (UAT).

4.6.1 Unit Testing

Unit tests were written for all backend service functions using Jest, a popular JavaScript testing framework. Key test cases included: verification that the merit list sorting algorithm produces the correct rank order for a given input set; testing that the automatic allocation engine correctly assigns rooms in merit rank order; verifying that the manual allocation API correctly rejects requests for already-allocated students or full rooms; and testing that password hashing and JWT token generation/validation work correctly.

4.6.2 Integration Testing

Integration tests were conducted using Postman to test the complete API endpoints. Each endpoint was tested with valid inputs, invalid inputs, missing parameters, and unauthorized access attempts. The tests verified that the API returns the correct HTTP status codes (200 for

success, 400 for bad request, 401 for unauthorized, 404 for not found, 500 for server error) and that the response body matches the expected format.

Table 4.5: Test Cases for Room Allocation Module

Test Case ID	Module	Test Description	Expected Result	Status
TC-01	Auth	Login with valid credentials	JWT token returned, 200 OK	PASS
TC-02	Auth	Login with invalid password	401 Unauthorized	PASS
TC-03	Merit List	Generate list with 10 students	Correct rank order	PASS
TC-04	Merit List	Generate list with empty DB	400 No students found	PASS
TC-05	Auto Alloc	Allocate 5 students, 5 rooms	All 5 allocated correctly	PASS
TC-06	Auto Alloc	Allocate with 0 available rooms	All waitlisted	PASS
TC-07	Manual Alloc	Assign room to allocated student	400 Already allocated	PASS
TC-08	Manual Alloc	Assign full room to student	400 Room at capacity	PASS
TC-09	Room Mgmt	Add new room with valid data	Room created, 201 Created	PASS
TC-10	Room Mgmt	Reduce capacity below occupancy	400 Invalid operation	PASS

4.6.3 User Acceptance Testing

User acceptance testing was performed with a simulated group of five admin users (faculty members) and ten student users. Participants were asked to perform a series of predefined tasks on the system and provide feedback using a structured evaluation form. The tasks included logging in, generating a merit list, performing automatic allocation, manually assigning a room, and viewing allocation status.

The results of the UAT were overwhelmingly positive. All participants successfully completed the tasks without assistance. The average task completion time for administrators was under 3 minutes for the full allocation cycle. Students found the merit list and allocation status pages

Hostel Merit Lists

Download gender-wise and year-wise hostel merit lists.

Male - 1st Year

Total Students: 20

#1 kumar

CGPA: 0

#2 sunil

CGPA: 0

#3 jin

CGPA: 0

#4 jinu

Download PDF

Male - 2nd Year

Total Students: 0

No students found.

Download PDF

Male - 3rd Year

Total Students: 2

#1 virat

CGPA: 9

#2 rohit

CGPA: 7

Download PDF

Male - Final Year

Total Students: 0

No students found.

Download PDF

MBM Hostel

MENU

Dashboard

Students

Merit List

Fee Management

Rooms

Allocation Algo

Complaints

Notice Board

Mess Menu

Room Management

Monitor occupancy and room status across all blocks.

All

Boys

Girls

All Years

1 Year

2 Year

3 Year

4 Year

All

Available

Full

Assigned

A1-1

2 SLOT(S)

Type

Boys

Block

Aryabhata Hostel

Gender

Boys

Year

4 Year

0/2

A1-2

2 SLOT(S)

Type

Boys

Block

Aryabhata Hostel

Gender

Boys

Year

1 Year

0/2

A1-3

2 SLOT(S)

Type

Boys

Block

Aryabhata Hostel

Gender

Boys

Year

1 Year

0/2

A1-4

2 SLOT(S)

Type

Boys

Block

Aryabhata Hostel

Gender

Boys

Year

1 Year

0/2

36

H

MBM Hostel

MENU

Dashboard

Students

Merit List

Fee Management

Rooms

Allocation Algo

Complaints

Notice Board

Mess Menu

Room Allocation

Smart hostel room allocation

Seating Plan Configuration

ADMIN DECISION ONLY

1st Year

S

D

T

2nd Year

S

D

T

3rd Year

S

D

T

Final Year

S

D

T

1

2

3

Final

kumar

Male • General • CGPA:0

Manual Allocation

All

Select Student

No Student Selected

H

MBM Hostel

MENU

Dashboard

Students

Merit List

Fee Management

Rooms

Allocation Algo

Complaints

Notice Board

Mess Menu

Admin Dashboard

TOTAL STUDENTS

22

OCCUPANCY

0%

PENDING COMPLAINTS

16

NOTICES

14

NO ROOM

22

HOSTEL FEES PENDING

22

MESS FEES PENDING

22

Chapter 5: Conclusion and Future Work

5.1 Summary of Work Done

This seminar report has presented a comprehensive study of the concept of Digital Room Allocation and its practical implementation through the Hostel Management System. The work undertaken encompasses all phases of the software development lifecycle, from initial requirement analysis to final testing and validation.

The project successfully achieved all its stated objectives. A fully functional merit list generation module was developed that ranks students based on gender and academic year. An automatic room allocation engine was implemented that assigns rooms in strict merit rank order, ensuring that the highest-ranked students receive their room assignments first. An admin manual allocation interface was developed that provides administrators with full override capability, allowing them to assign any available room to any unallocated student directly. A room inventory management module was created that enables administrators to configure room quantities, types, and capacities dynamically. Finally, a student-facing dashboard was developed that provides real-time access to merit list rankings and allocation status.

The system was built using the MERN stack (MongoDB, Express.js, React.js, Node.js), which provided a cohesive and efficient development environment. The three-tier architecture ensured clean separation of concerns, making the system maintainable and scalable. The REST API design followed industry standards, enabling future integration with other institutional systems.

5.2 Achievements and Findings

The following key achievements and findings emerged from this project:

- The merit-based automatic allocation algorithm significantly reduces the administrative effort required for hostel room assignment. A manual process that previously took several days can now be completed in seconds.
- The dual-mode allocation system (automatic + manual) provides the best of both worlds: efficiency and fairness from the automatic merit-based engine, and flexibility and control from the manual admin interface.
- The use of MongoDB proved highly effective for this application. Its document-based model accommodated the varied and sometimes sparse nature of student and room data without requiring rigid schema definitions.
- JWT-based authentication and role-based access control (RBAC) provided a secure and scalable mechanism for managing user sessions and permissions.

- The system's real-time dashboard significantly improved the visibility and transparency of the allocation process for both administrators and students.
- Testing revealed that the system handles edge cases (no rooms available, no merit list, double-allocation attempts) gracefully, without data corruption or system crashes.

5.3 Limitations

Despite its many strengths, the current implementation of the Hostel Management System has several limitations that should be acknowledged:

14. Email Notification: The current system does not send email notifications to students when they are allocated a room. Students must log in to the system to check their status.
15. Single-Hostel Scope: The system is designed for a single hostel. Multi-hostel or multi-campus scenarios are not currently supported.
16. Advanced Merit Criteria: The current merit system only considers year and gender. Academic CGPA, distance from home, or special category reservations are not factored in.
17. Mobile Application: The system is a web-based application and has not been developed as a native mobile app, though the React frontend is responsive.
18. Payment Integration: Fee collection for hostel charges is not integrated into the current system.
19. Room Preference: Students cannot express preferences for specific rooms or roommates through the system.

5.4 Future Enhancements

Based on the limitations identified above and the suggestions received during user acceptance testing, the following future enhancements are proposed for subsequent versions of the Hostel Management System:

Table 5.1: Proposed Future Enhancements

Enhancement	Description	Priority
Email & SMS Notifications	Automated alerts to students upon room allocation	High
Advanced Merit Criteria	Incorporate CGPA, reservation categories, and distance from home	High
Online Fee Payment	Integration with payment gateway for hostel fee collection	High
Room Preference Form	Allow students to submit room and roommate preferences	Medium
Multi-Hostel Support	Extend to manage multiple hostels under one admin portal	Medium

Enhancement	Description	Priority
Mobile Application	Develop native Android/iOS apps for student access	Medium
Grievance Portal	Allow students to raise complaints and allocation disputes	Medium
Analytics Dashboard	Visual analytics on occupancy trends, allocation history	Low
Biometric Integration	Link room access with biometric attendance system	Low
Alumni Room Release	Automated de-allocation at end of semester/academic year	High

5.5 Final Conclusion

The Hostel Management System represents a meaningful contribution to the digitization of administrative processes at educational institutions. By replacing a manual, opaque, and error-prone room allocation process with a digital, merit-based, and transparent system, the project directly addresses a real-world problem faced by thousands of residential students at institutions like MBM University, Jodhpur.

The technical implementation of the system demonstrates the practical applicability of modern web technologies — React.js, Node.js, Express.js, and MongoDB — in solving real institutional challenges. The project also illustrates the value of good software engineering practices: modular design, proper API architecture, role-based security, transactional database operations, and comprehensive testing.

The seminar topic of Digital Room Allocation is not merely an academic exercise. As India's higher education system expands with increasing student enrollments and new hostel constructions, the need for scalable, reliable, and fair digital allocation systems will only grow. This project provides a solid foundation upon which such systems can be built, extended, and deployed.

In conclusion, the Hostel Management System achieves its goal of digitizing room allocation through a merit-based, administrator-controlled, and student-transparent platform. The system is functionally complete, well-tested, and ready for pilot deployment. Future enhancements, particularly email notifications, advanced merit criteria, and multi-hostel support, will further strengthen the system and expand its applicability across diverse institutional contexts.

REFERENCES

- [1] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- [2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.
- [3] Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.
- [4] Chodorow, K. (2019). MongoDB: The Definitive Guide (3rd ed.). O'Reilly Media.
- [5] Banks, A., & Porcello, E. (2020). Learning React (2nd ed.). O'Reilly Media.
- [6] Cantelon, M., Harter, M., Holowaychuk, T. J., & Rajlich, N. (2014). Node.js in Action. Manning Publications.
- [7] Hipp, D. R., Kennedy, D., & Mistachkin, J. (2021). SQLite Documentation. Retrieved from <https://www.sqlite.org/docs.html>
- [8] MongoDB Inc. (2024). MongoDB Documentation. Retrieved from <https://www.mongodb.com/docs/>
- [9] Facebook Inc. (2024). React.js Official Documentation. Retrieved from <https://react.dev/>
- [10] OpenJS Foundation. (2024). Node.js Documentation. Retrieved from <https://nodejs.org/en/docs/>
- [11] JWT.io. (2024). JSON Web Tokens Introduction. Retrieved from <https://jwt.io/introduction>
- [12] OWASP Foundation. (2023). OWASP Top Ten Web Application Security Risks. Retrieved from <https://owasp.org/www-project-top-ten/>
- [13] Fowler, M. (2018). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- [14] Richardson, L. (2013). RESTful Web APIs. O'Reilly Media.
- [15] MBM University. (2024). Hostel Allotment Rules and Regulations – Academic Year 2024-25. Department of Student Affairs, MBM University, Jodhpur.
- [16] Koskela, L. (2013). Practical TDD and Acceptance TDD for Java Developers. Manning Publications.
- [17] Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.
- [18] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.